

Análise e Projeto de Software Orientado a Objetos

Revisão dos conceitos do paradigma orientado a objetos

Leonardo S. Victorio

Faculdade de Computação - UFMS



- 1 O Conceito de Abstração
- 2 Objetos e Classes
- 3 Atributos, Métodos e Encapsulamento
- 4 Mensagens e Relacionamentos
- 5 Herança e Polimorfismo

Introdução à Abstração

- Um dos principais conceitos da orientação a objetos, usado na análise de domínio para representar um escopo.
- Consiste em observar um recorte da realidade e representar suas entidades, ações e caracterizações.
- Foca no que é necessário para a aplicação atingir seu objetivo.



O Mecanismo de Foco

- A abstração deixa de lado os aspectos que são irrelevantes para aquele contexto específico.
- Na modelagem, busca-se focar em quais atividades são realizadas, como são feitas e quem as realiza.
- Evita a tentativa incorreta de mapear absolutamente todas as propriedades do mundo real.



Subjetividade na Abstração

- Um mesmo cenário pode ser abstraído de formas completamente diferentes dependendo do observador.
- Exemplo: Um software de música modelado por um usuário comum será diferente daquele modelado por um músico ou produtor.
- Depende diretamente do conhecimento do analista e do objetivo final do sistema.



- Não existe uma única resposta ou modelagem correta para um problema.
- Existem inúmeras modelagens que podem representar fielmente a mesma situação.
- Um modelo só estará incorreto se estiver incompatível com o mundo real ou não atender ao objetivo.



Tudo é Objeto

- Na orientação a objetos, tudo (ou quase tudo) é tratado como um objeto.
- Um objeto é o resultado direto da aplicação do mecanismo de abstração sobre uma entidade do mundo real.
- Pode ser uma entidade concreta, uma ideia, um sentimento ou uma informação volátil.



- Possuem características internas e realizam operações inerentes ao sistema observado.
- São entidades “vivas”: são criados (instanciados), modificados, executam ações e podem ser destruídos ao longo da execução.



Identificando Objetos: Exemplo Prático

- Cenário: Software de rastreamento de ônibus em um sistema viário urbano.
- O “Ponto de Ônibus” é um elemento do mundo real que precisa ser abstraído como um objeto no sistema.
- O analista define como ele será representado e quais ações poderá realizar.



Ponto de Ônibus
+ Localização: String
+ Horários: List<Horário>
+ Endereço: String
+ temAbrigo: Boolean
+



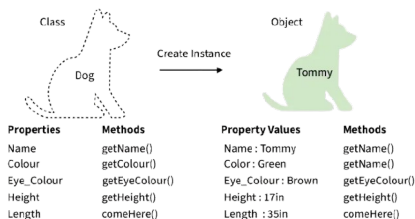
O Conceito de Classe

- Muitas vezes as definições de classe e objeto se confundem, mas são conceitos distintos.
- Uma classe é um agrupamento estrutural que define o mesmo conjunto de atributos e métodos.
- Atua como um molde para agrupar objetos instanciados no sistema.



Diferença entre Classe e Objeto

- A classe define os tipos de dados sem necessariamente definir os valores.
- O objeto é a instância real de uma classe.
- Ele atribui valores próprios aos atributos que foram desenhados na classe.
- Exemplo: A definição “Ponto de Ônibus” é a classe. O ponto da “Avenida Central, 123” é o objeto.



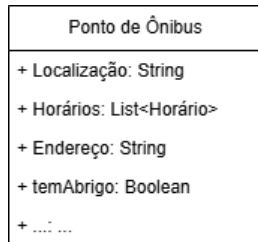
Atributos: O Estado do Objeto

- Buscam representar os elementos de dados que dão características exclusivas a um objeto.
- Podem ser entendidos como variáveis que descrevem o estado atual daquele objeto.
- Representam valores armazenados que podem ser lidos, alterados ou excluídos.



Definição de Atributos

- Quem define os atributos necessários é o analista, de acordo com o objetivo do sistema.
- Exemplo: Uma classe “Ponto de Ônibus” pode ter localização e lista de horários.
- Se o sistema exige saber se há proteção contra chuva, adiciona-se o atributo “tem abrigo” (booleano).



Métodos: O Comportamento

- Também conhecidos como operações ou funções, são as ações que o objeto pode realizar.
- Geralmente atuam alterando o estado do objeto e manipulando suas informações.
- Descrevem uma sequência de ações pré-definidas.



Características dos Métodos

- Junto aos atributos, os métodos compõem a totalidade dos objetos e não podem ser isolados deles.
- Exemplo: Na classe “Ponto de Ônibus”, podemos ter métodos como “mostrar horário” e “atualizar horário”.

```
class PontoDeOnibus:
    def __init__(self, localizacao: str, endereco: str, tem_abrigo: bool, horarios: list = None):
        self.localizacao = localizacao
        self.endereco = endereco
        self.tem_abrigo = tem_abrigo
        self.horarios = horarios if horarios is not None else []

    def mostrar_horarios(self):
        """Exibe a lista de horários disponíveis no ponto."""
        if not self.horarios:
            print(f"Não há horários registrados para o ponto em: {self.endereco}")
        else:
            print(f"Horários para o ponto ({self.localizacao}):")
            for h in self.horarios:
                print(f"- {h}")

    def adicionar_horario(self, novo_horario: str):
        """Adiciona um novo horário à lista existente."""
        self.horarios.append(novo_horario)
        print(f"Horário {novo_horario} adicionado com sucesso!")
```



O Conceito de Encapsulamento

- Uma característica vital que consiste em esconder e proteger detalhes de implementação entre entidades.
- É alcançado gerenciando a visibilidade (privacidade) dos atributos do objeto.
- O acesso direto à informação interna por outros objetos deve ser evitado.



A Interface Pública

- Em geral, atributos são configurados como privados e métodos como públicos.
- Os métodos públicos compõem o que chamamos de “Interface Pública da Classe”.
- A única forma segura de acessar ou modificar um atributo privado é acionando um método fornecido pelo próprio objeto.



Vantagens do Encapsulamento

- Restringe a visibilidade de regras de negócio internas.
- Aumenta de forma drástica a manutenibilidade das operações.
- Facilita a modularização natural e a responsabilização que a classe fornece no projeto.



A Comunicação via Mensagens

- É o mecanismo que permite a comunicação e a interoperabilidade entre diferentes objetos no sistema.
- Na prática, o envio de uma mensagem corresponde ao acionamento (ou chamada) da execução de um método.
- Essa comunicação respeita rigorosamente o encapsulamento através da interface pública.



Estrutura de uma Mensagem

- É composta por um nome simbólico chamado de **seletor** (o próprio nome da operação).
- O **objeto emissor** solicita a execução da mensagem.
- O **objeto receptor** é aquele que recebe a solicitação e executa a operação de fato.



Argumentos na Mensagem

- A mensagem pode carregar um conjunto de argumentos (ou parâmetros) necessários para a execução.
- A ordem dos parâmetros geralmente deve respeitar a assinatura do método (embora dependa da linguagem).
- Exemplo Prático: Um emissor envia a mensagem “abastecer” passando os argumentos “diesel” e “50” para o objeto receptor “meu_onibus”.

```
public class Main {  
    public static void main(String[] args) {  
        // Criar um objeto da classe Onibus  
        Onibus meuOnibus = new Onibus();  
  
        // Chamar o método abastecer com parâmetros (enviar mensagem)  
        double porcentagem = meuOnibus.abastecer("Diesel", 50.0);  
  
        // Exibir o retorno do método  
        System.out.println("Porcentagem do tanque após abastecimento: " + porcentagem + "%");  
    }  
}
```



- Objetos frequentemente possuem relações expressas entre si.
- Ocorrem quando uma característica de uma entidade é complexa demais para ser representada apenas por um tipo primitivo.
- Exemplo: Um “Ônibus” precisa ter conhecimento dos locais de parada. Uma lista de tipos primitivos não basta para definir completamente um “Ponto de Ônibus”.

O Relacionamento de Associação

- É o tipo mais comum de relacionamento.
- Indica que uma determinada classe ou objeto conhece outro, estabelecendo uma conexão estrutural.
- Na implementação, significa que um dos atributos do objeto “Ônibus” é do tipo (ou lista do tipo) da classe “Ponto de Ônibus”.



O Mecanismo de Herança

- Permite definir uma nova classe a partir de uma classe já existente.
- A classe original é chamada de **superclasse**, e a nova classe derivada é a **subclasse**.
- A subclasse herda automaticamente todos os métodos e atributos da superclasse.



A Extensão via Herança

- A subclasse tem a liberdade de adicionar novos atributos e métodos exclusivos.
- Pode também alterar (sobrescrever) características recebidas da superclasse se houver necessidade.
- Exemplo: Uma superclasse “Veículo” pode derivar as subclasses “Ônibus”, “Van” e “Ônibus Elétrico”.

```
# Superclasse (Pai)
class Veiculo:
    def __init__(self, capacidade_tanque: float):
        self.__capacidade_tanque = capacidade_tanque
        self.__quantidade_atual = 0.0

    @property
    def capacidade_tanque(self):
        return self.__capacidade_tanque

    def abastecer(self, tipo_combustivel: str, quantidade: float) -> float:
        self.__quantidade_atual += quantidade
        if self.__quantidade_atual > self.__capacidade_tanque:
            self.__quantidade_atual = self.__capacidade_tanque

        percentagem = (self.__quantidade_atual / self.__capacidade_tanque) * 100
        print(f"Abastecendo com {tipo_combustivel}: {quantidade}. Total: {percentagem}%")
        return percentagem

# Subclasses (Filhas)
class Onibus(Veiculo):
    def __init__(self, capacidade_tanque=200.0):
        # O super() chama o construtor da classe pai
        super().__init__(capacidade_tanque)
        self.tipo = "Ônibus Urbano"

class Van(Veiculo):
    def __init__(self, capacidade_tanque=80.0):
        super().__init__(capacidade_tanque)
        self.tipo = "Van de Passageiros"

class OnibusEletrico(Veiculo):
    def __init__(self, capacidade_bateria=500.0):
        super().__init__(capacidade_bateria)
        self.tipo = "Ônibus Elétrico"

    def abastecer(self, tipo_energia: str, quantidade_kwh: float) -> float:
        print(f"Conectando ao carregador de {tipo_energia}...")
```

- **Generalização:** Observar da subclasse para a superclasse. Podemos afirmar que todo ônibus é um veículo.
- **Especialização:** Observar da superclasse para a subclasse. Um veículo pode ser do tipo Van.
- Na generalização, uma variável do tipo superclasse pode receber instâncias de suas subclasses.

Hierarquia e Execução de Métodos

- Ao receber uma mensagem, o objeto procura o método correspondente primeiro em sua própria classe.
- Se o método não for encontrado ali, a busca avança para cima, procurando na superclasse na hierarquia.
- Esse processo se repete de baixo para cima até a operação ser encontrada para execução.



Introdução ao Polimorfismo

- É a capacidade que um objeto instanciado possui de assumir formas de comportamento diferentes.
- Complementa o uso de hierarquias de herança permitindo forte flexibilidade ao código.



Polimorfismo de Sobrescrita (Override)

- A subclasse altera o comportamento interno de um método herdado mantendo exatamente a mesma assinatura.
- Exemplo: “Ônibus Elétrico” altera a execução do método “abastecer” de “Veículo” por não utilizar diesel, mas energia.
- O método da superclasse continua acessível se for chamado via operador como ‘super’.



Polimorfismo de Sobrecarga (Overload)

- Uma única classe apresenta múltiplos métodos utilizando o mesmo seletor (nome).
- Cada um desses métodos possui uma **assinatura diferente** (quantidade ou tipo de parâmetros recebidos).
- A decisão sobre qual método específico executar é tomada dinamicamente em tempo de execução com base nos argumentos enviados.

